03 · 总体架构设计（SAD）

项	内容
文档编号	DT-PdM-03
依据	TOGAF 10 阶段 B（业务架构）/ C（信息系统架构：数据+应用）/ D（技术架构）；C4 模型；4A 架构
版本	v0.1

---

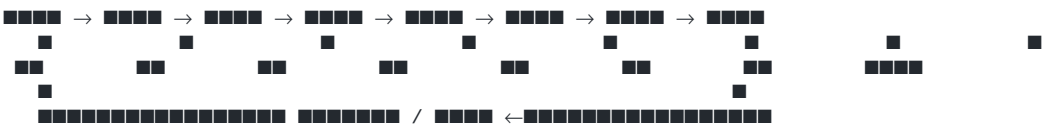
1. 架构原则（Architecture Principles）

ID	原则	理由	违反示例（反模式）
P1	建议不指令	系统不参与安全联锁，不承担控制责任	Agent 直接向 PLC 写设定值
P2	证据可回放	工业场景要"说错了谁负责"	只输出"置信度 87%"无依据
P3	降级优于崩溃	脏数据、断网、掉线是常态	一个通道失效导致整条链停摆
P4	协议无关	工厂协议碎片化，上层不应感知	诊断代码里写 `if protocol == 'modbus'`
P5	人机共决	关键决策必须留人工确认	自动创建工单并派工
P6	单点可替换	开源组件生命周期不可控	算法与 supOS/TDengine 深度耦合
P7	配置优于编码	新设备接入不应发版	每接一台设备改代码重新部署
P8	默认最小权限	等保三级 + 工控安全	Agent 拥有全库读写权限

---

2. 业务架构（TOGAF 阶段 B）

2.1 价值链

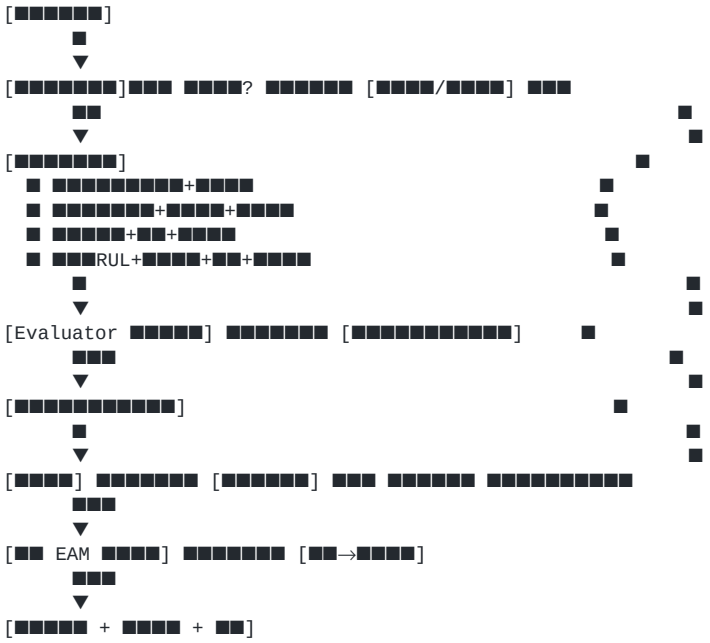


2.2 业务能力地图

L1 能力	L2 能力	系统支撑
设备资产管理	台账、关键度分级、BOM	M1
状态监测	采集、波形、趋势	M1
故障诊断	特征分析、故障模式匹配	M2
健康管理	HI、劣化趋势	M2
寿命预测	RUL、OOD 识别	M3
维护决策	会诊、窗口优化、经济性	M5

L1 能力	L2 能力	系统支撑
维护执行	工单、备件、派工	M5 + EAM
知识管理	案例库、判据库、技能库	M8
治理合规	审计、权限、模型治理	M8

2.3 业务流程（决策主流程，BPMN 简化）



---

3. 应用架构（TOGAF 阶段 C-应用 / 4A 之 AA）

3.1 分层视图

3.2 服务清单（微服务边界）

服务	职责	技术	独立部署	备注
`ingest-gateway`	协议接入、归一化	Go / Neuron		边缘

服务	职责	技术	独立部署	备注
`asset-registry`	统一命名空间、元数据	Python FastAPI		中心
`ts-writer`	时序写入、降采样	Go		高吞吐
`feature-engine`	特征提取（时域/频域/包络）	Python + NumPy/SciPy		可边缘化
`diagnosis-engine`	SD/HA，规则+模型	Python		插件化
`prognosis-engine`	RUL	Python		插件化
`knowledge-service`	故障模式库、案例库	Python + PG/Neo4j		—
`agent-runtime`	Harness 运行时	Python asyncio		核心
`mcp-server`	MCP 工具暴露	Python MCP SDK		无状态
`decision-service`	会诊编排、方案卡	Python		核心
`alert-service`	分级路由、升级、抑制	Go		—
`integration-eam`	EAM/MES 适配	Python		防腐层
`report-render`	HTML/PDF 报告	Node/Python		—
`audit-service`	审计链	Go		哈希链

边界原则： 一个服务一个数据库 schema，禁止跨库 JOIN；跨域通信用事件。

3.3 关键架构决策（ADR 摘要）

ADR	决策	备选	理由	后果
ADR-001	自研 Harness，不直接用 LangGraph/CrewAI	LangGraph、CrewAI、Hermes Agent	工业场景需要"权限+审计+Stop Hook"一等公民；框架抽象会隐藏关键控制点	开发量 +15%，可控性 +100%
ADR-002	时序库选 TDengine	InfluxDB、TimescaleDB	国产、压缩率高（10:1+）、单机吞吐强、支持降采样	生态较弱，需自建运维
ADR-003	诊断引擎插件化（策略模式 + 注册表）	单体	模型会换（LimiX 许可风险）	需定义稳定接口
ADR-004	会诊用"并行 fan-out + 独立 Evaluator"	单一 Agent 串行	防自说自话；可解释	token 成本 ×4（可接受）
ADR-005	边缘用 K3s，不用 OpenClaw 生产	OpenClaw	高频变更不适合产线	失去"自主写代码"能力，用配置替代
ADR-006	原始波形留厂，只传特征	全量上云	合规 + 带宽	云端无法重算，需边缘可回放
ADR-007	MCP 作为唯一对外工具协议	自定义 REST	生态兼容、可被任意 LLM 客户端调用	需做 MCP 网关鉴权
ADR-008	报告用 HTML（非 PDF 优先）	PDF	可交互（缩放频谱）、可嵌入证据	归档需另存 PDF

> ADR 全文在 Ph1 落地为 `docs/adr/ADR-00x.md`，每条含"背景-决策-后果-状态"。

4. 数据架构（TOGAF 阶段 C-数据 / 4A 之 DA）

4.1 数据分层

层	内容	存储	保留
ODS 原始层	原始报文、原始波形	对象存储 + 边缘盘	90 天（波形）/ 1 年（报文）
DWD 明细层	清洗后时序、对齐后多源	TDengine	3 年
DWS 汇总层	小时/日聚合、特征宽表	TDengine + PG	5 年
ADS 应用层	HI、RUL、告警、方案卡	PG	5 年
知识层	故障模式、案例、判据、技能	PG + 图库 + 向量	永久

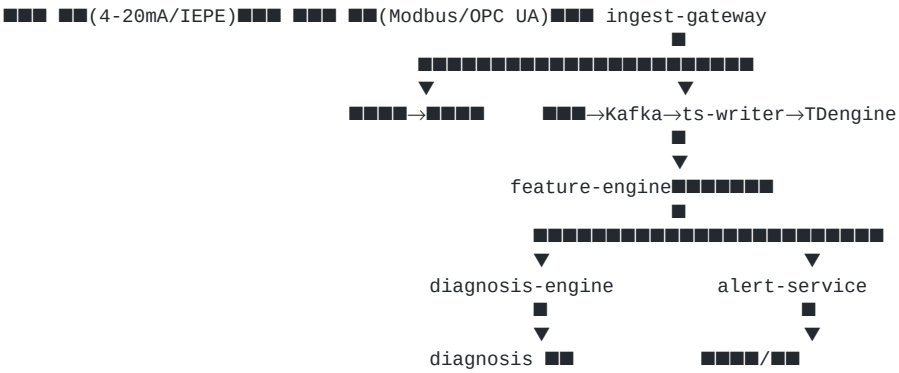
4.2 统一命名空间（核心设计）

```
{site}.{area}.{asset}.{component}.{point}.{quantity}
■■SZ01.■■■■.KA-101.■■■■■.DE_H.vibration.acc_rms

■■■
· ■■■■■■■■6 ■■■
· asset ■■■■■■ P&ID ■■■
· point ■■ ISO 13373 ■■■■■DE/NDE × H/V/A■
· quantity ■■ UCUM ■■■■
```

为什么重要： 这是"协议无关"的落地手段。上层永远只认这个路径，换 PLC 不影响任何上层逻辑。

4.3 数据流



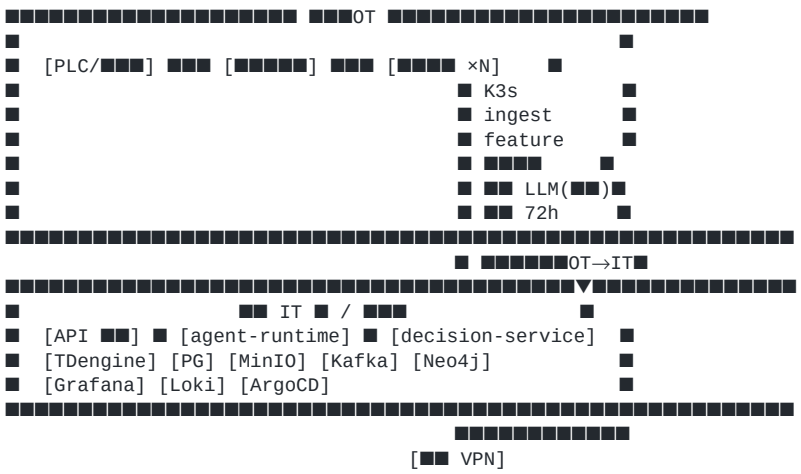
5. 技术架构（TOGAF 阶段 D / 4A 之 TA）

5.1 技术栈选型

层	选型	版本策略	备选	选型理由
OS	Ubuntu 22.04 LTS（中心）/ 麒麟 V10（信创）	LTS	统信 UOS	生态 + 信创双轨
容器	Docker + containerd	固定 minor	Podman	成熟
编排	K8s（中心）/ K3s（边缘）	1.30+	—	边缘资源占用低

层	选型	版本策略	备选	选型理由
GitOps	ArgoCD	—	Flux	声明式、可回滚
消息	EMQX（MQTT）+ Kafka	—	NATS	MQTT 工业标配；Kafka 做回放
时序库	TDengine 3.x	—	TimescaleDB	压缩比、写入吞吐
关系库	PostgreSQL 16	—	达梦	JSONB、pgvector 一体
图库	Neo4j Community	—	NebulaGraph	故障传播路径查询
向量库	pgvector	—	Milvus	少一个组件
缓存	Redis 7	—	—	—
对象存储	MinIO	—	—	S3 兼容、可本地
后端	Python 3.12（算法/Agent）+ Go 1.22（网关/采集）	—	—	各取所长
Agent	自研 Harness（asyncio）+ MCP SDK	—	—	ADR-001
模型网关	LiteLLM Proxy	—	自研	多模型路由、成本核算
前端	React 18 + TypeScript + ECharts/AntV	—	Vue	图表能力强
观测	OpenTelemetry + Prometheus + Grafana + Loki	—	—	标准
CI/CD	Harness.io（主）+ GitHub Actions（备）	—	Jenkins	见 docs/09

5.2 部署拓扑（云边协同）



关键设计：

- 数据闸单向（OT→IT），反向仅允许运维白名单端口。
- 边缘节点自治：中心挂了不影响本地诊断与告警。

- 模型更新走"边缘拉取 + 灰度"，不推。

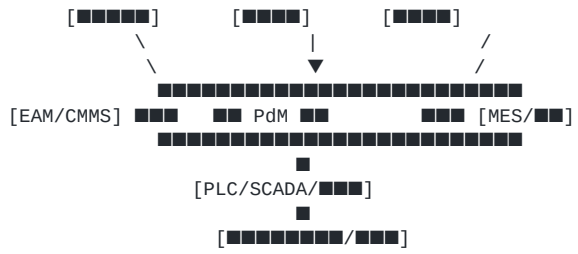
5.3 容量规划（20 台设备基线）

组件	规格	数量
边缘节点	8C/32G/512G SSD	3（按车间）
中心 K8s	16C/64G × 3	1 集群
TDengine	8C/32G/1T SSD	3（副本）
GPU（可选）	L4 / 4090 × 1	1（本地推理）
存储	10 TB 可用	—

---

6. C4 模型（关键视图）

6.1 Context（系统上下文）



6.2 Container（容器视图）

见 §3.1 分层视图；容器间通信：同步用 gRPC/REST，异步用 Kafka/MQTT。

6.3 Component（组件视图 · agent-runtime）

```
agent-runtime
■■■ LoopController        ■■■■■■■■■
■■■ ContextBuilder        ■■■■■■■■■■■/■■/■■■■
■■■ ToolRegistry          ■■■■■■■■
■■■ PermissionGate        allow/ask/deny ■■
■■■ SandboxExecutor       ■■■■■■■■■■+CPU■■■■
■■■ MemoryStore           ■■■■■
■■■ HookChain             pre/post/stop hooks
■■■ Tracer                OTel span
■■■ CostMeter             token/■■■■■
```

6.4 Code（代码视图）

见 `docs/09 §3` 仓库结构。

---

7. 架构治理（TOGAF 阶段 G 前置）

机制	内容	频率
架构评审委员会	架构师 + 设备专家 + 安全 + 平台负责人	每个 Gate
ADR 机制	每个重大决策一页纸，纳入版本库	随时
技术债登记	记录已知妥协与偿还计划	每迭代
合规检查	对照 docs/12 标准矩阵逐条核	每阶段
架构契约测试	服务间接口契约测试（Pact 类）	每次 CI

架构例外流程： 任何违反 P1 – P8 原则的设计，必须提交"例外申请"，说明理由、风险、补偿措施、失效期，由架构委员会批准。没有例外流程的原则等于没有原则。